

Array Analytics

Arrays, Kadane's Maximum Subarray, Equilibrium Index · Mentor-Led Lab

Developed by TalenciaGlobal

Difficulty ★★★ Intermediate-Advanced

Problem

Two classic array questions, one pass each. Given an integer array, find the maximum contiguous subarray sum and the first equilibrium index (where the values to the left sum to the same as the values to the right).

You are given starter code with two functions. `max_subarray_sum` is almost complete but has a bug, and `equilibrium_index` is an empty stub. Fix the bug and implement the stub.

What the Program Reports

- Max Subarray Sum: the largest sum of any contiguous subarray (Kadane's algorithm).
- Equilibrium Index: the first index i where the sum of the elements before i equals the sum of the elements after i (excluding i), or -1 if none exists.

Input / Output Format

Input: Line 1: n . Line 2: n integers (which may be negative).

Output: Two lines: Max Subarray Sum, Equilibrium Index.

Examples

Example 1

Input: `arr = [-3, 5, -2, 5, -3]`

Output:

```
Max Subarray Sum: 8
Equilibrium Index: 2
```

Explanation: The best subarray is `[5, -2, 5]` = 8; at index 2 the left sum (`-3+5=2`) equals the right sum (`5-3=2`).

Example 2

Input: `arr = [1, 2, 3, 4, 6]`

Output:

```
Max Subarray Sum: 16
Equilibrium Index: 3
```

Explanation: All positive, so the whole array sums to 16; at index 3 the left (1+2+3=6) equals the right (6).

Constraints

- $1 \leq n \leq 10^5$; values fit in a 64-bit integer.
- The maximum subarray is non-empty (at least one element).
- Return the first equilibrium index, or -1 if there is none.

Your Task

- Task 1 - Fix the bug. `max_subarray_sum` is flagged with a comment, but the bug's location is not given. Find the single bug so the answer is correct even when a fresh start beats extending the current run.
- Task 2 - Implement `equilibrium_index` from scratch in a single pass using prefix sums.

Starter Code

Begin from the code below. One function carries a single bug (flagged by a comment, with no hint to its location); the other is an empty stub you must implement.

Python

```
# Lab 3 - Array Analytics: Maximum Subarray + Equilibrium Index
import sys

# This function contains a BUG. Find it and fix it.
# It should return the maximum contiguous subarray sum (Kadane's algorithm).
def max_subarray_sum(arr):
    max_so_far = arr[0]
    current = arr[0]
    for i in range(1, len(arr)):
        current = current + arr[i]
        max_so_far = max(max_so_far, current)
    return max_so_far

# TODO: Implement this function from scratch.
# Return the first index where the sum of elements to its left equals the
# sum of elements to its right (excluding the index itself), or -1.
def equilibrium_index(arr):
    return 0

def main():
    data = sys.stdin.read().split()
    n = int(data[0])
    arr = [int(data[1 + i]) for i in range(n)]
    print("Max Subarray Sum: " + str(max_subarray_sum(arr)))
    print("Equilibrium Index: " + str(equilibrium_index(arr)))

main()
```

C++

```
// Lab 3 - Array Analytics: Maximum Subarray + Equilibrium Index
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;
// This function contains a BUG. Find it and fix it.
long long maxSubarraySum(const vector<long long>& a) {
    long long best = a[0], cur = a[0];
    for (size_t i = 1; i < a.size(); i++) { cur = cur + a[i]; best = max(best, cur); }
    return best;
}
// TODO: Implement this function from scratch.
// Return the first index where the sum to its left equals the sum to its
// right (excluding the index), or -1.
int equilibriumIndex(const vector<long long>& a) {
    return 0;
}
int main() {
    int n; cin >> n;
    vector<long long> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];
    cout << "Max Subarray Sum: " << maxSubarraySum(a) << "\n";
    cout << "Equilibrium Index: " << equilibriumIndex(a) << "\n";
    return 0;
}
```

Java

```
// Lab 3 - Array Analytics: Maximum Subarray + Equilibrium Index
import java.util.*;
public class Main {
    // This function contains a BUG. Find it and fix it.
    static long maxSubarraySum(long[] a) {
        long best = a[0], cur = a[0];
        for (int i = 1; i < a.length; i++) { cur = cur + a[i]; best = Math.max(best, cur); }
        return best;
    }
    // TODO: Implement this function from scratch.
    // Return the first index where the sum to its left equals the sum to its
    // right (excluding the index), or -1.
    static int equilibriumIndex(long[] a) {
        return 0;
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        long[] a = new long[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextLong();
        System.out.println("Max Subarray Sum: " + maxSubarraySum(a));
        System.out.println("Equilibrium Index: " + equilibriumIndex(a));
    }
}
```